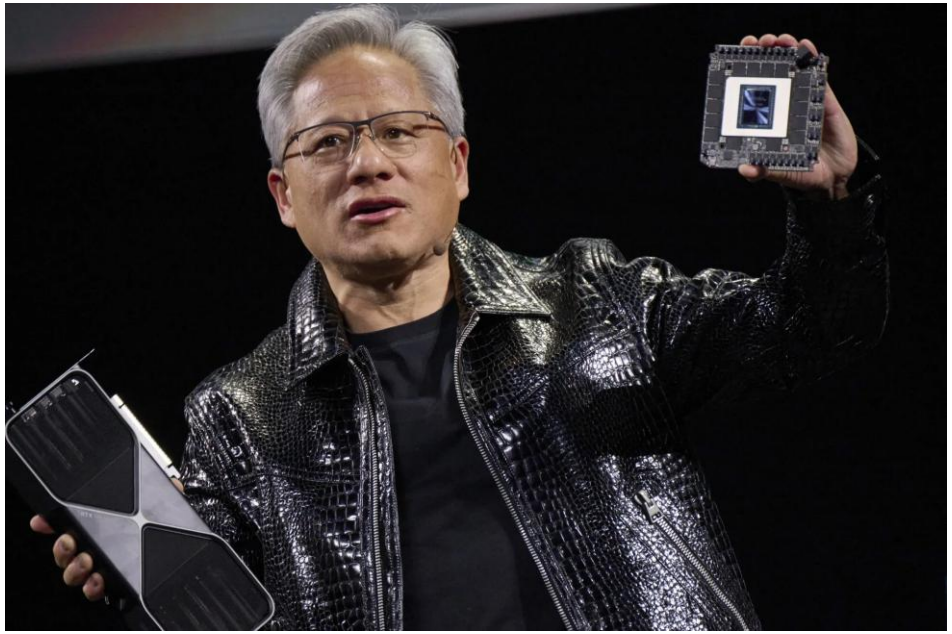
Neuromorphic Computing with RISC-V Manycore

喵喵 @ Wuhan RISC-V Meetup

Academic Slops

~~Neuromorphic Computing~~ with RISC-V Manycore

喵喵 @ Wuhan RISC-V Meetup



Research
RISC-V

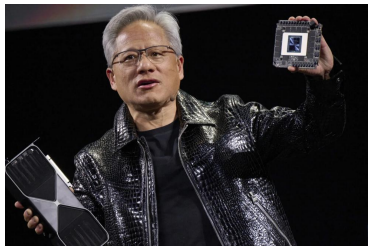
Use
RISC-V

Neuromorphic Computing

n. is a computing approach inspired by the human brain's structure and function.[1][2] It uses artificial neurons to perform computations, mimicking neural systems for tasks such as perception, motor control, and multisensory integration.[3] These systems, implemented in analog, digital, or mixed-mode VLSI, prioritize robustness, adaptability, and

Research
RISC-V

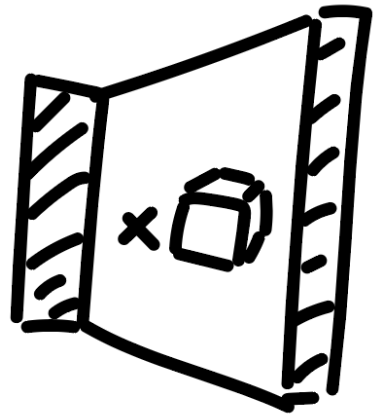
Use
RISC-V

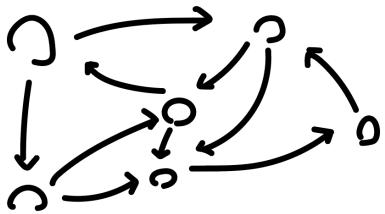


Neuromorphic Computing

SNN
Spiking
脉冲神经网络

- High parallelism, but with frequent communication & synchronization.
- Unpredictable and sparse memory access

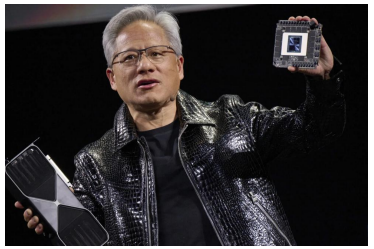




Research RISC-V

Abuse RISC-V

Use RISC-V



Why RISC-V?

Abusing RISC-V (The ISA)

Feels great!

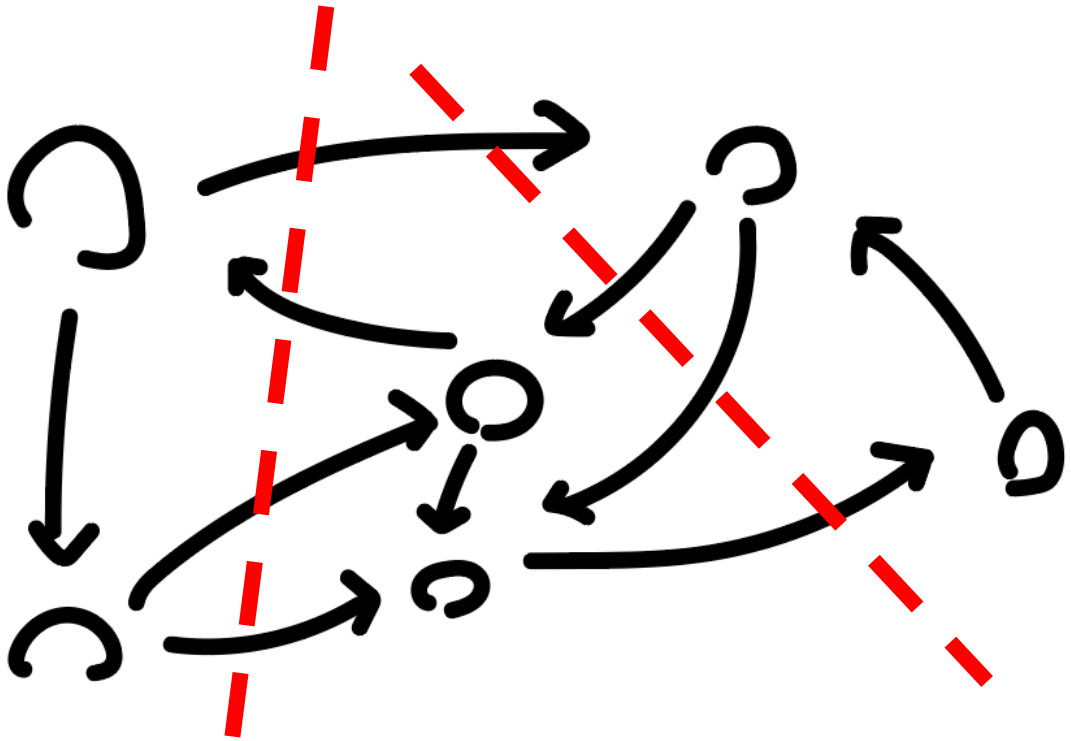
Not really, I love her with all my heart

Combination of extensions provides flexibility

1. F-in-X (`Zfinx`)
2. `Zaamo` vs. `Zalrsc`
3. RoCC

Abusing RISC-V (The ISA)

F-in-X extension (Zfinx)



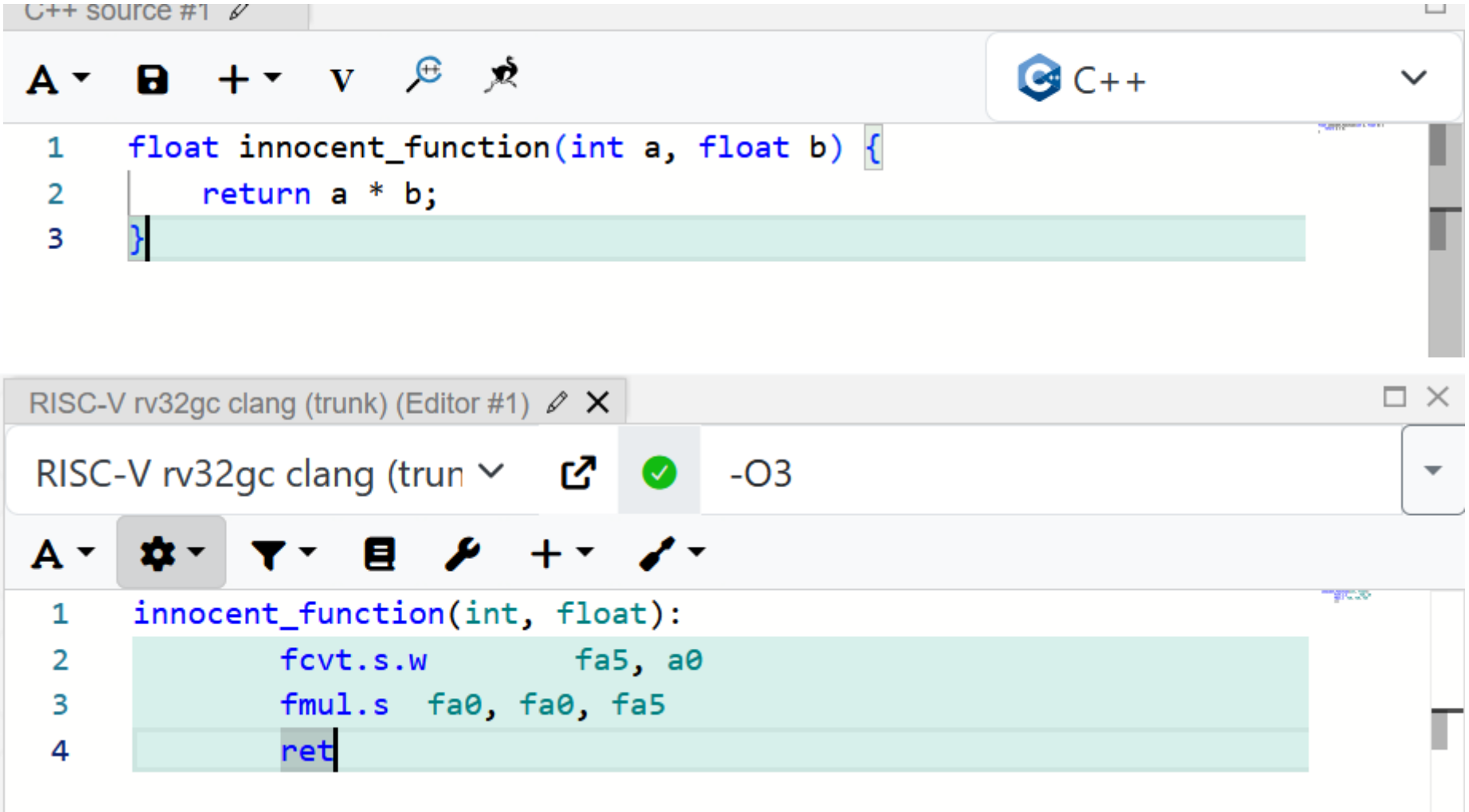
Abusing RISC-V (The ISA)

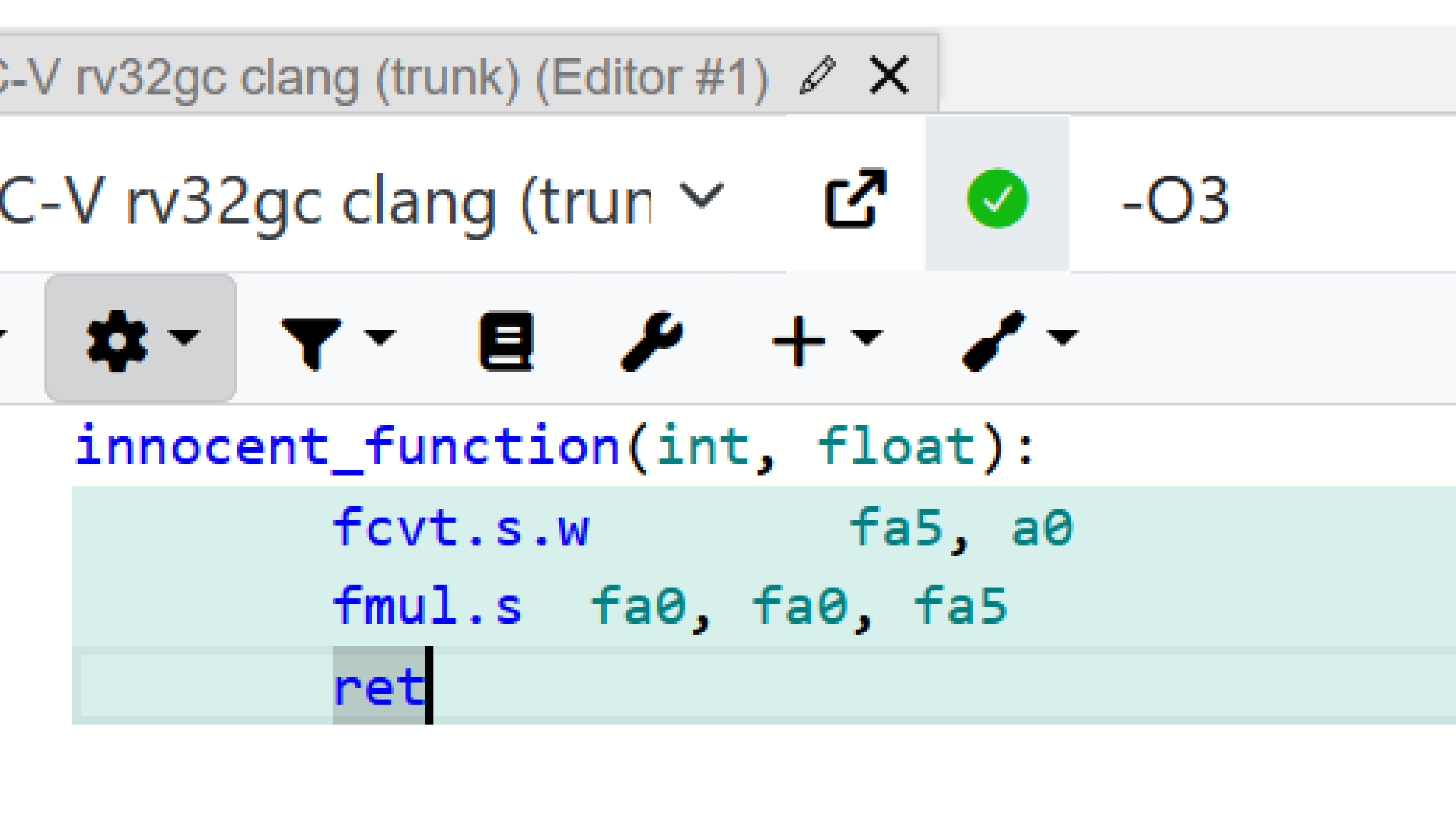
F-in-X extension (Zfinx)

- 256 – 512 Cores

- **F-in-X**: FP shares Int register file

- Core Area: **~85000um²** @ 1GHz + 28nm
- +SMT(double register file): **~92000um²**
- ➔ w/o Zfinx: **~110000um²** (+15%)


C++ source #1
C++
1 float innocent_function(int a, float b) {
2 return a * b;
3 }
RISC-V rv32gc clang (trunk) (Editor #1)
RISC-V rv32gc clang (trun
-O3
1 innocent_function(int, float):
2 fcvt.s.w fa5, a0
3 fmul.s fa0, fa0, fa5
4 ret


C-V rv32gc clang (trunk) (Editor #1)
C-V rv32gc clang (trun
-O3
innocent_function(int, float):
fcvt.s.w fa5, a0
fmul.s fa0, fa0, fa5
ret

[RISCV][CodeGen] Support Zfinx codegen

Closed Public

Authored by sunshaoce on Apr 20 2023, 9:48 PM.

Details

Reviewers
craig.topper
jrtc27
asb
rogfer01
luismarques
frasercrmck
reames
kito-cheng
arcbbb

Commits rGfe558efe71c1: [RISCV][CodeGen] Support Zfinx codegen

SUMMARY

This patch was split from D122918 . Co-Author: @liaolucy @realqhc

Abusing RISC-V (The ISA)

AMO Only (+Zaamo -Zalrsc)

- A extension: Atomic instructions for multi-core cooperation

- LR/SC: coherent fabric, local cache → `Zalrsc`
- AMO: 🚌 support, remote cache → `Zaamo`

- w/o local cache, custom mesh fabric

Abusing RISC-V (The ISA)

AMO Only (+Zaamo -Zalrsc)

- State:
  - OpenSBI builds on both `Zalrsc and/or Zaamo`
  - Uboot builds with `Zaamo` by default

  - No emulation for LR/SC on top of AMO: ABA problem.
  - Needs double width CAS…
  - Allocate static space in core context!

Abusing RISC-V (The ISA)

RoCC

- A dirty and fast way to quickly spin up a custom extension
  - No modification to the decoder
  - No modification to the uArch

  - Don’t need to write a core yourself!

Speaking of Writing a Core…

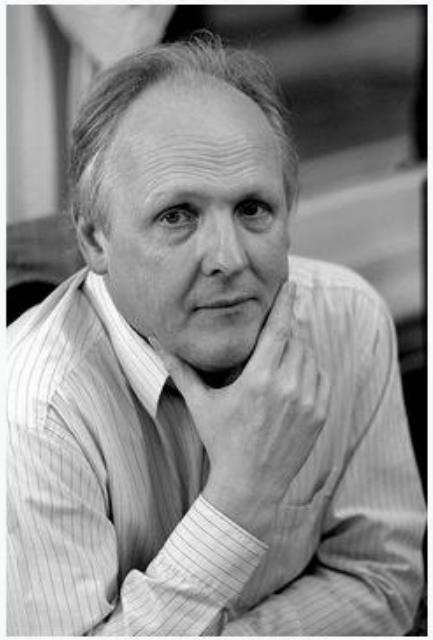
SpiNNaker: spiking neural network architecture

The SpiNNaker 1 million core machine assembled at the University of Manchester

Developer Steve Furber

Known for
Acorn Computers
ARM architecture
BBC Micro
SpiNNaker[9]
Human Brain Project[10]
ARM System–on–Chip Architecture[11]

Steve Furber
CBE FRS FREng



Furber in 2009

Born	Stephen Byram Furber 21 March 1953 (age 73)[6] Manchester, England[7]
Education	Manchester Grammar School
Alma mater	University of Cambridge (BA, MMath, PhD)[6][8]
Known for	Acorn Computers ARM architecture BBC Micro SpiNNaker[9] Human Brain Project[10] *ARM System–on–Chip Architecture*[11]

SpiNNaker: spiking neural network architecture

The SpiNNaker 1 million core machine assembled at the University of Manchester

Developer	Steve Furber
Product family	Manchester computers
Type	Neuromorphic
Released	2019
CPU	ARM968E-S @ 200 MHz
Memory	7 TB
Successor	SpiNNaker 2[1]
Website	SpiNNaker

- Uses off-the-shelf ARM processors
- Interrupts as scheduling primitive
- No CUDA-like local scratchpad

Open ISA

Free-as-in-Freedom

Open RISC-V Implementatio

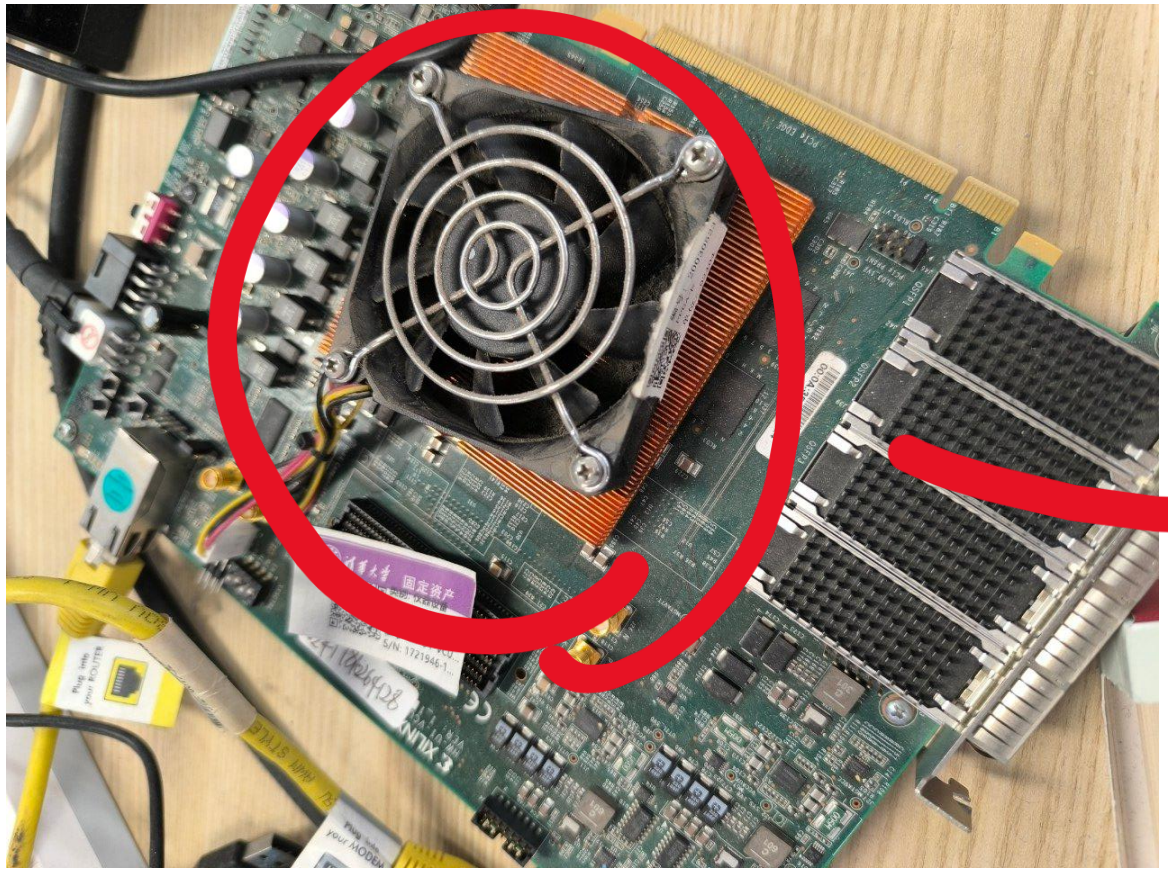
Explore open RISC-V implementatio

Name	
AUK-V-Aethia	AUK-V RV32I CPU.
CV32E40P	In-order 4-stage R (PULP-Platform).
CVA6	CORE-V CVA6, an of booting Linux.

https://github.com/risc

Open ISA

Bring your own core!



meow-chip/MeowV64
Proudly powered by
@jiegec

Open ISA

Bring your own core!

Commits on Nov 29, 2023

Basic
CircuitCoder committed on Nov 29, 2023

Commits on Nov 19, 2023

Fixing various bugs
CircuitCoder committed on Nov 19, 2023

Simulator
CircuitCoder committed on Nov 19, 2023

Simulator skeleton
CircuitCoder committed on Nov 19, 2023

Br
CircuitCoder committed on Nov 19, 2023

Basic pipeline and ICache
CircuitCoder committed on Nov 19, 2023

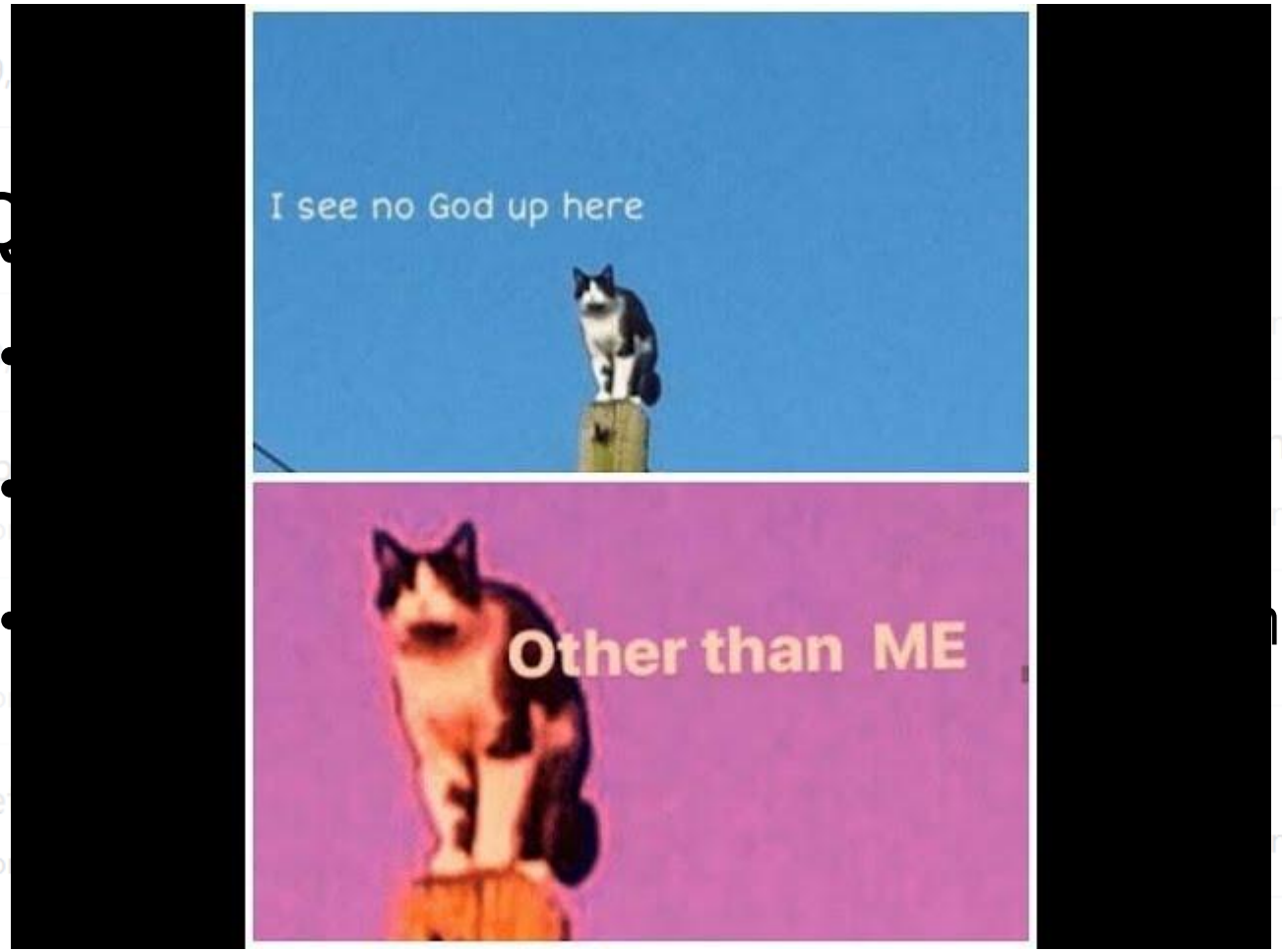
Commits on Nov 18, 2023

Init
CircuitCoder committed on Nov 18, 2023

Open ISA

Bring your own core!

- C
-
-
- *



Design & Implementation

NoC & Mem

- Two separated NoCs:
  - 2D mesh for P2P data and sendin
  - Reduced ring + 1-to-N broadcast

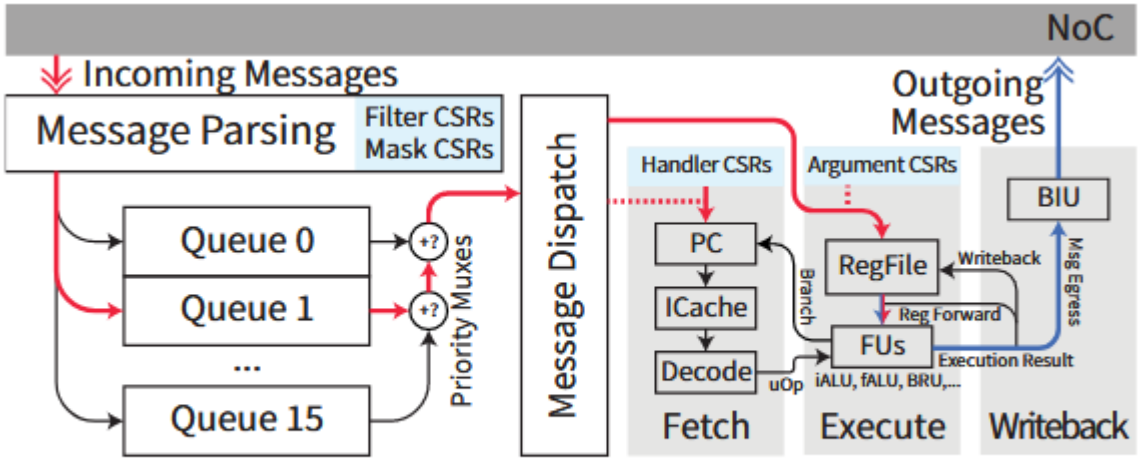


Fig. 5: Hardware Pipeline Overview

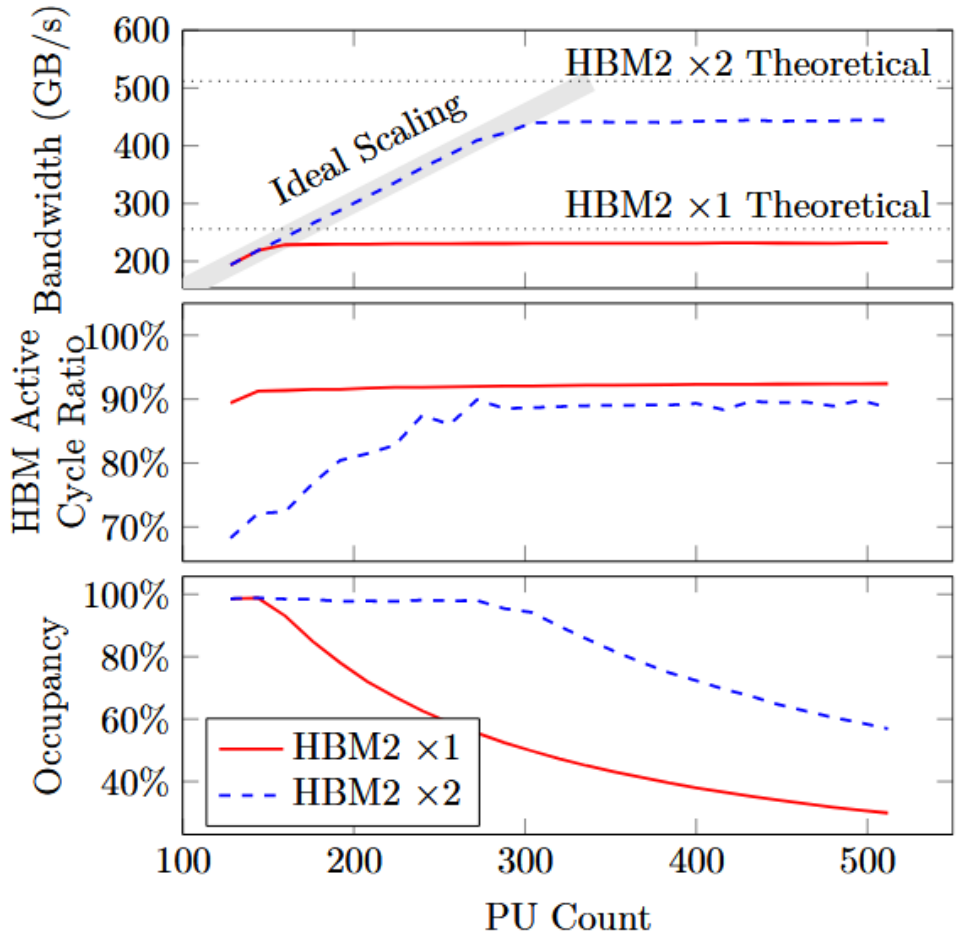


Fig. 7: Scalability of PU Counts

Design & Implementation

NoC & Mem

- Customized fabric is somewhat unavoidable. Bus, protocol and NoC are highly coupled with the ISA:
  - RVWMO being MCA → TileLink's Grant/GrantAck ordering
  - RISC-V has AMO → TileLink's encoding
  - TileLink's message priorities → Router VC/PC prioritization

  - Splitting the protocol and the link may be a last hope.

Design & Implementation

Scheduling

- No preemptive scheduling: no context switch overhead.
- We even use that to manage SMT threads! So it’s by definition not different harts.

- Needs A LOT of work to guarantee deadlock free execution.

Design & Implementation

Summary

- Homebrew core for maximum performance per mm^2/W
- Custom fabric for domain-specific memory access pattern: broadcast, highly zoned.
- Designed for communication-mediated scheduling

Thank you!

Sidenotes

RISC-V for research